

Pre-Norm, Post-Norm, and Stability

Same parts, different order · video 2:28

This lesson answers one question: does it matter whether the normalization comes before or after each block? **Post-norm** (the original Transformer, BERT) normalizes the stream itself: $x = \text{norm}(x + \text{block}(x))$. **Pre-norm** (GPT-2, Llama, Qwen) normalizes only what the block reads: $x = x + \text{block}(\text{norm}(x))$. At a low learning rate they train equally well; at three times the learning rate, post-norm stalls within ten steps unless the learning rate is warmed up, while pre-norm trains either way.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Two orders	▶ 0:08-0:54	2	☐
2. Learning rate, and warmup	▶ 0:54-1:41	2	☐
3. Where it goes wrong, and why pre-norm won	▶ 1:41-2:19	2	☐

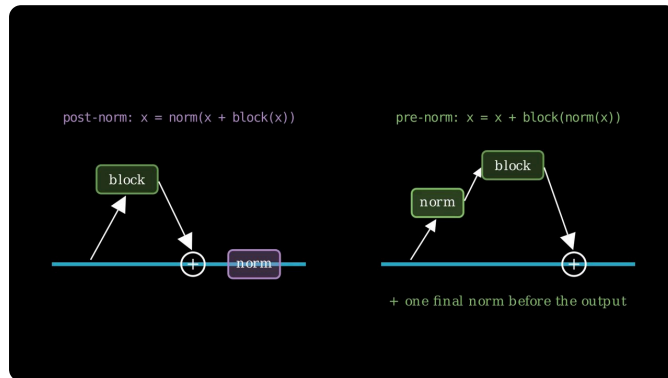
Before you start: Deep Dive episodes 13 (residual stream) and 14 (normalization). Code: `code/d16_pre_post_norm` (PyTorch only; ten 12-layer tiny models, about an hour on a CPU).

CONCEPT 1 OF 3

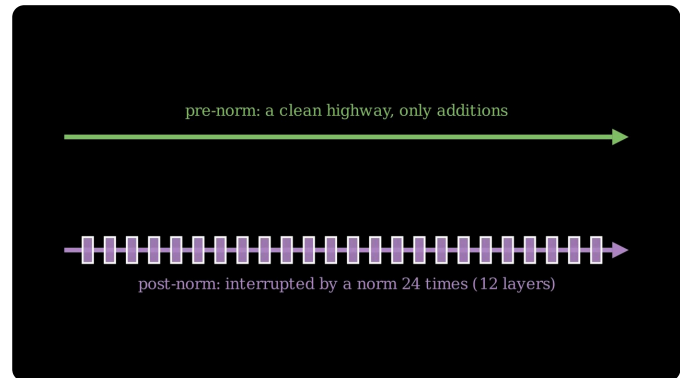
Two orders

▶ Video 0:08-0:54

The original Transformer and BERT put the normalization **after** each block: add the block's output to the stream, then normalize the stream itself. GPT-2, Llama and Qwen put it **before**: normalize only what the block reads, and add its output to the stream untouched. Pre-norm then needs one final norm before the output layer.



▶ 0:36 Post-norm: $x = \text{norm}(x + \text{block}(x))$. Pre-norm: $x = x + \text{block}(\text{norm}(x))$, plus one final norm.



▶ 0:53 Pre-norm keeps the residual stream a clean highway; post-norm interrupts it 24 times in 12 layers.

In pre-norm, the residual stream of episode 13 is a clean highway: nothing but additions from the embedding to the end. In post-norm, every layer's sum passes through a norm, so the direct path is interrupted 24 times in a 12-layer model (after each attention and each MLP).

KEY IDEA

Pre-norm normalizes the block's input; post-norm normalizes the stream. Only pre-norm keeps the identity path intact.

Check 1: Two orders

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

How many times is the residual stream normalized in a 32-layer post-norm transformer (one attention and one MLP per layer)?

1.2 TRUE OR FALSE

"In pre-norm, the residual stream is never normalized before the output layer."

True ☐ False ☐

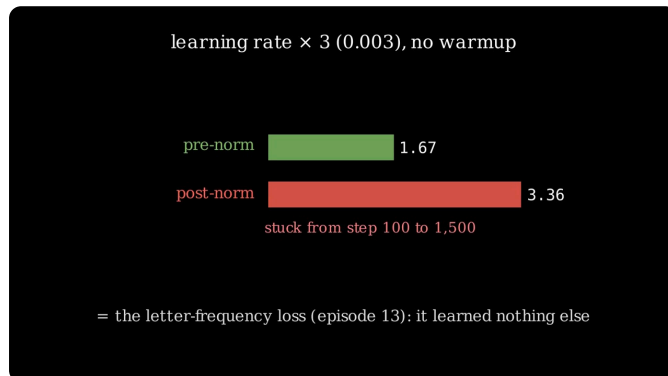
CONCEPT 2 OF 3

Learning rate, and warmup

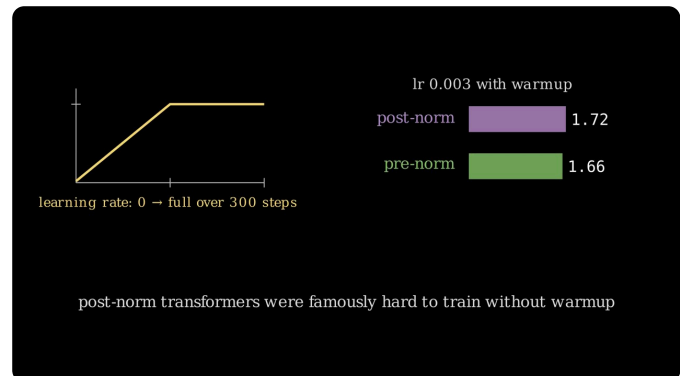
▶ Video 0:54–1:41

A 12-layer tiny GPT, 1,500 steps. Validation loss at learning rate 0.001: pre-norm **1.67**, post-norm **1.68**. No difference. At three times the learning rate (0.003): pre-norm 1.67 again; post-norm stuck at **3.36** from step 100 to the end, the letter-frequency loss of episode 13. It learned nothing else.

The classic fix is **warmup**: start the learning rate near zero and raise it linearly over the first 300 steps. With warmup at 0.003, post-norm trains: **1.72**; pre-norm reaches 1.66. Post-norm transformers were famously hard to train without warmup.



▶ 1:21 Learning rate 0.003, no warmup: pre-norm 1.67; post-norm stuck at 3.36 from step 100 to 1,500.



▶ 1:39 With 300 warmup steps: post-norm 1.72, pre-norm 1.66.

KEY IDEA

Post-norm needs warmup at higher learning rates; pre-norm does not, though warmup can still help.

Check 2: Learning rate, and warmup

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

With 300 warmup steps and a peak learning rate of 0.003, what is the learning rate at step 150 (linear warmup, as in the code)?

2.2 MULTIPLE CHOICE

Which run failed to learn?

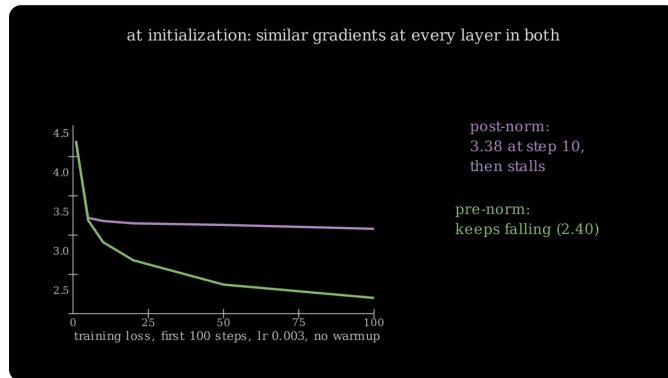
- A. Pre-norm, lr 0.003, no warmup
- B. Post-norm, lr 0.001, no warmup
- C. Post-norm, lr 0.003, no warmup
- D. Post-norm, lr 0.003, with warmup

CONCEPT 3 OF 3

Where it goes wrong, and why pre-norm won

▶ Video 1:41-2:19

At initialization, in this small model, both versions get similar gradients at every layer (the last layer's gradient is 0.9 times the first's in both). The trouble comes with the **first updates**: within ten steps post-norm's training loss stalls at **3.38** and never recovers (3.28 at step 100), while pre-norm keeps falling (3.11 at step 10, 2.40 at step 100). Warmup makes those first updates small.



▶ 1:58 First 100 steps at lr 0.003: post-norm stalls at 3.38 by step 10; pre-norm keeps falling to 2.40.

```
1 # pre-norm (GPT-2, Llama, Qwen): normalize what the block reads
2 x = x + attn(ln1(x))
3 x = x + mlp(ln2(x))
4
5 # post-norm (original Transformer, BERT): normalize the stream itself
6 x = ln1(x + attn(x))
7 x = ln2(x + mlp(x))
```

▶ 2:17 In code, the whole difference is where the norm sits.

That is why modern models use pre-norm: it trains stably at higher learning rates and with less careful tuning, even when the network gets very deep. In code the difference is where the norm sits: `x = x + block(norm(x))` versus `x = norm(x + block(x))`.

KEY IDEA

Instability shows up in the first steps; pre-norm and warmup both protect them.

Check 3: Where it goes wrong, and why pre-norm won

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

Post-norm's loss stalled at about the letter-frequency value. What does that suggest the model ended up predicting?

3.2 HANDS-ON CODE

Try it yourself. In `pre_post_norm.py`, train post-norm at lr 0.003 with a shorter warmup of 50 steps. Does it still train?

```
print(train(pre=False, lr=3e-3, warmup=50))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Two orders · rewatch ▶ 0:08-0:54

1.1 64. Two norms per layer, each applied to the stream itself.

1.2 False. Pre-norm adds one final norm before the output layer; inside the layers the stream itself is never normalized.

2. Learning rate, and warmup · rewatch ▶ 0:54-1:41

2.1 About 0.0015. Halfway through the warmup: $0.003 \times 151 / 300 \approx 0.00151$ (the code uses $(\text{step} + 1) / \text{warmup}$).

2.2 C. Stuck at 3.36; the other three reached 1.67–1.72.

3. Where it goes wrong, and why pre-norm won · rewatch ▶ 1:41-2:19

3.1 Roughly the same distribution of characters everywhere, ignoring the context. A context-free prediction scores exactly the letter-frequency (unigram) loss, 3.35 here.

3.2 Yes: 2.40 / 1.90 / 1.75 at 100 / 500 / 1,500 steps, between no warmup (stuck at 3.36) and 300 steps (1.72). Even a short warmup protects the first updates; how short it can be depends on the model and learning rate.